Anomaly Detection in Industrial IoT Sensor Streams Using Autoencoders: Project Report

Variant 3E: Metro Train Air Compressor | Team 19 | CM701PC Deep Learning, CSE (AI and ML), MGIT (Autonomous), Hyderabad | Guide: Dr. M. Rudra Kumar

Nov 5, 2026 · @hbk

Certificate

This is to certify that the project report titled **"Anomaly Detection in Industrial IoT Sensor Streams Using Autoencoders (Variant 3E: Metro Train Air Compressor)"** is a bonafide record of work carried out by Team 19, listed below, in the Department of CSE (Artificial Intelligence and Machine Learning), Mahatma Gandhi Institute of Technology (Autonomous), Hyderabad, for the course CM701PC Deep Learning / CM751PC Deep Learning Lab, under the guidance of Dr. M. Rudra Kumar.

Roll number	Name
23261A6648	Srihari Harkar
23261A6626	M. Tharun Reddy
23261A6602	A. Prabhas

Project guide: Dr. M. Rudra Kumar (signature and date to be added)

Head of the Department: (name, signature and date to be added)

Declaration

We declare that this report is our own work. The experiments were run by us on the MetroPT-3 dataset, and every result reported here was taken from the output files of that run. No result was copied from another source or filled in by estimate. The use of AI tools is disclosed at the end of this report.

Srihari Harkar (23261A6648), M. Tharun Reddy (23261A6626), A. Prabhas (23261A6602)

Acknowledgement

We thank our project guide, Dr. M. Rudra Kumar, for his guidance and for setting the evaluation targets that shaped this work. We also thank the Department of CSE (AI and ML) and MGIT for the facilities, and the authors of the MetroPT-3 dataset for making it public.

Abstract

Train air compressors fail rarely, and labelled fault data is scarce, so we built an unsupervised early-warning system on the UCI MetroPT-3 dataset. A Dense autoencoder and an LSTM autoencoder were trained only on normal readings from February 2020, using windows of 30 readings with stride 3. Reconstruction error was the anomaly score. Two thresholds were chosen on the labelled evaluation period (March to August 2020): one maximising F1 and one minimising a cost of ₹4,00,000 per missed fault and ₹2,000 per false alarm.

At the cost-aware threshold both models detected all four reported failures (recall 1.00). The Dense model raised 1.07 false alarms per 24 hours with a mean detection delay of 50.4 minutes and an estimated cost of ₹3,92,000; the LSTM raised 1.23 per 24 hours with a delay of 73.5 minutes and a cost of ₹4,52,000. Both met the recall and false-alarm targets, and neither met the 5-minute delay target. Thresholds were tuned on the test period and only four failures exist, so the figures are optimistic.

1 Introduction

1.1 Background

A metro train's air production unit supplies compressed air for brakes, doors and suspension, so an unexpected compressor failure can take a train out of service. Maintenance based on fixed intervals either services healthy units too often or misses faults between visits. Condition-based maintenance instead watches live sensor data (pressure, oil temperature, motor current, valve states) and acts when behaviour changes.

Faults are rare, and a plant seldom holds enough labelled examples to train a conventional classifier. An autoencoder sidesteps this: it is trained on normal data alone, learns to reconstruct it, and reconstructs unfamiliar patterns poorly. The reconstruction error then serves as an anomaly score.

1.2 Problem Statement

Given multivariate telemetry from a metro air compressor, detect abnormal operation early enough for maintenance to act, without training on labelled fault data and without flooding engineers with false alarms.

1.3 Objectives

1. Detect abnormal operating conditions in metro air-compressor sensor streams.
2. Train autoencoders using normal operating data only.
3. Compare a Dense autoencoder and an LSTM autoencoder.
4. Minimise false alarms while detecting actual faults.
5. Reduce the delay between fault occurrence and detection.
6. Select an operating threshold using both F1-score and cost-sensitive analysis.

The assignment sets four targets: fault-event recall of at least 0.85, fewer than 2 false alarms per 24 hours, detection delay under 5 minutes (or within 3 readings where sampling is slower than once a minute), and no labelled fault data in training.

1.4 Scope and Assumptions

- Only the public MetroPT-3 dataset is used, and only its four reported air-leak failures count as faults.
- February 2020 is assumed to be normal operation. The dataset authors recommend the first month for training, but we did not verify that no early degradation is present.
- Fault labels are used only to tune thresholds and to evaluate, never to train.
- Costs follow the assignment: ₹4,00,000 per missed fault event and ₹2,000 per false-alarm episode.
- The work is offline analysis of recorded data. No real-time deployment is attempted.

1.5 Overall System Architecture

The system is a pipeline of seven stages:

1. Load and sort the MetroPT-3 records by timestamp.
2. Clean the data and scale each sensor using statistics from the training period only.
3. Cut the stream into overlapping windows of 30 readings with stride 3.

4. Train the Dense and LSTM autoencoders on normal windows from February 2020.
5. Score every later window by its mean squared reconstruction error.
6. Choose a best-F1 threshold and a cost-aware threshold.
7. Merge flagged windows into alarm episodes and measure event recall, false alarms per day, detection delay and cost.

1.6 Report Organisation

Chapter 2 reviews related work. Chapter 3 describes the dataset and data preparation. Chapter 4 presents the two models and the training protocol. Chapter 5 explains scoring, thresholds and metrics. Chapter 6 reports and discusses results, and Chapter 7 concludes. The appendix records how to reproduce the run.

2 Literature Review

2.1 Anomaly Detection

Anomaly detection finds observations that do not follow the expected pattern of the data. Chandola, Banerjee and Kumar [3] group methods into classification-based, nearest-neighbour, statistical, clustering and reconstruction-based approaches. Supervised methods need labelled anomalies, which are rare in industry, so semi-supervised methods that learn only from normal data are common.

2.2 Autoencoders for Unsupervised Representation Learning

An autoencoder is a network trained to copy its input to its output through a narrow bottleneck, which forces it to learn a compact representation [5, 6]. If the bottleneck is too wide the network can learn an identity mapping and reconstruct anything, anomalies included. Trained on normal data, it reconstructs normal inputs well and unfamiliar inputs less well, so the reconstruction error separates the two.

2.3 LSTM Networks and Temporal Modeling

Long Short-Term Memory networks [4] use gated memory cells to carry information across many time steps. In a sequence autoencoder an LSTM encoder compresses a window into a latent vector and an LSTM decoder rebuilds the sequence. Malhotra et al.

[7] used this idea for multi-sensor anomaly detection. A Dense autoencoder sees a flattened window and has no built-in notion of order, whereas the LSTM models order directly.

2.4 Deep Anomaly Detection and Predictive Maintenance

The MetroPT-3 dataset and its first analysis come from Davari et al. [1, 2], who applied deep-learning anomaly detection to the air production unit of metro trains in Porto. Their work established the dataset, the recommended training month and the reported failure intervals that this project uses for evaluation.

2.5 Research Gap and Project Position

Published comparisons often report window-level scores. A maintenance team cares about events: was each failure caught, how late, and how many warnings were false. This project therefore adds event-level evaluation (recall over fault events, false-alarm episodes per day, detection delay) and a cost-sensitive threshold beside the usual best-F1 threshold, and compares a Dense and an LSTM autoencoder under identical conditions.

3 Dataset, Requirements, and Data Preparation

3.1 Dataset Description

MetroPT-3 is a UCI Machine Learning Repository dataset of 1,516,948 time-stamped readings from the air production unit of a metro train [1]. The signals include pressure sensors, oil temperature, motor current and digital states of the compressor and valves. The UCI page reports four air-leak failures, which we use as the only fault labels.

Item	Value
Source	UCI MetroPT-3 (DOI 10.24432/C5VW3R)
Records	1,516,948
Normal training period	1 Feb 2020 to 1 Mar 2020

Item	Value
Evaluation period	1 Mar 2020 onward (about 184 days of scored windows)
Reported failures	4 air-leak intervals

Failure interval	Duration
18 Apr 2020 00:00 to 19 Apr 00:00	24 h
29 May 2020 23:30 to 30 May 06:00	6.5 h
5 Jun 2020 10:00 to 7 Jun 14:30	52.5 h
15 Jul 2020 14:30 to 19:00	4.5 h

The exact number of input channels and the median sampling interval are printed in the first lines of the run log; the pipeline uses every column except the timestamp and the CSV's index column.

3.2 System Requirements

- **Software:** Python 3, TensorFlow/Keras, scikit-learn, pandas, NumPy, Matplotlib (see requirements.txt).
- **Hardware:** the experiment was run on Google Colab. A GPU is recommended because the LSTM autoencoder is slow on a CPU.
- **Data:** about 208 MB of CSV downloaded from UCI by download_data.py.
- **Reproducibility:** fixed random seed 42 for Python, NumPy and TensorFlow.

3.3 Industrial IoT Sensor Data Flow

Sensors on the compressor are sampled by the train's monitoring unit and logged with a timestamp. For this project the logged stream is replayed offline, in time order, through the pipeline of Section 1.5. In a live deployment the same scaler and model would score each new window as it arrives and send alarms to the maintenance team.

3.4 Preprocessing Pipeline

1. Parse timestamps, sort by time, drop duplicate timestamps and rows with missing or non-numeric values.
2. Select the training period, 1 February to 1 March 2020.
3. Split that period chronologically: the first 80% fits the model and the last 20% is validation.
4. Fit a standard scaler (zero mean, unit variance) on the 80% training portion only, then apply it unchanged to validation and evaluation data. This avoids leaking later behaviour into the scaler.
5. Mark each evaluation reading as faulty if its timestamp lies inside a reported failure interval.

3.5 Sliding-Window Time-Series Processing

The scaled stream is cut into overlapping windows of 30 consecutive readings, advancing 3 readings at a time. A window is scored at the timestamp of its last reading, which makes detection delay well defined. In the evaluation period a window is labelled positive if any of its readings lies in a failure interval.

Evaluation windows	Count
Positive (overlap a failure)	10,026
Normal	423,997
Total	434,023

Only about 2.3% of evaluation windows are positive, so accuracy would be a misleading metric and we report precision, recall, F1 and event-level measures instead.

3.6 Exploratory Data Analysis

The run did not produce per-sensor plots, so we limit this section to what the outputs show. Class imbalance is strong (2.3% positive windows). The reconstruction-error trace over the evaluation period (Figure 6.3) shows that most windows score far below the threshold, with isolated spikes reaching several thousand scattered through March to August. A per-sensor exploration of those spikes is left to future work.

4 Methodology and Autoencoder Models

4.1 Unsupervised Learning Formulation

Let x be a scaled window of 30 readings. A model f is trained to minimise the difference between x and f(x) over normal windows only. At test time the anomaly score of a window is how badly f reconstructs it. No fault label enters training, so the system can in principle flag fault types it has never seen.

4.2 Autoencoder Structure and Latent Representation

An autoencoder has an encoder that maps the window to a small latent vector z, and a decoder that maps z back to a window. Both of our models use a 16-unit latent vector. The narrow latent layer is what prevents the network from copying its input unchanged and makes reconstruction error informative.

4.3 Dense Autoencoder

The baseline flattens each 30-reading window into one vector and passes it through fully connected layers.

Stage	Layer	Units	Activation
Input	Flatten	30 × channels	none
Encoder	Dense	128	ReLU
Encoder	Dense	64	ReLU
Bottleneck	Dense	16	ReLU
Decoder	Dense	64	ReLU
Decoder	Dense	128	ReLU
Output	Dense, reshaped to 30 × channels	30 × channels	linear

4.4 LSTM Autoencoder

The sequence model keeps the time axis. An LSTM encoder reads the window and its final state becomes the latent vector, which is repeated across 30 steps and decoded by a second LSTM stack.

Stage	Layer	Units
Encoder	LSTM, returns the full sequence	64
Bottleneck	LSTM, returns the final state	16
Bridge	RepeatVector to 30 steps	n/a
Decoder	LSTM, returns the full sequence	16
Decoder	LSTM, returns the full sequence	64
Output	TimeDistributed Dense, linear	channels

Because the LSTM can model order within the window, it could in principle capture temporal patterns that a flattened Dense input represents less naturally. Whether it did so here is assessed in Chapter 6.

4.5 Reconstruction Loss

Training minimises the mean squared error between a window and its reconstruction, and the same quantity is the anomaly score:

```
s(x) = \frac{1}{L \cdot F} \sum_{t=1}^{L} \sum_{j=1}^{F} \left( x_{t,j} - \hat{x}_{t,j} \right)^2
```

where L = 30 is the window length, F is the number of sensor channels, x is the scaled window and x-hat its reconstruction.

4.6 Model Configuration and Training Protocol

Setting	Value
Window length / stride	30 readings / 3

Setting	Value
Optimiser	Adam, learning rate 0.001
Loss	Mean squared error
Batch size	256
Maximum epochs	20
Early stopping	validation loss, patience 4, restore best weights
Learning-rate schedule	halve on plateau, patience 2
Training / validation split	80% / 20% in time order within February 2020
Random seed	42

Both models trained for the full 20 epochs, so early stopping did not trigger.

Model	Final training loss	Final validation loss
Dense AE	0.0813	0.0631
LSTM AE	0.1658	0.1147

5 Anomaly Scores, Thresholds, and Evaluation

5.1 Anomaly Score Calculation

Each evaluation window is passed through the trained model and its score is the mean squared reconstruction error of Section 4.5. Because scores depend on the scaled units of each model, thresholds are not comparable between the Dense and the LSTM model; only the resulting alarms are.

5.2 Threshold-Based Detection

A window is flagged as anomalous when its score reaches or exceeds a threshold. Consecutive flagged windows are merged into one alarm episode, so a long fault produces one alarm rather than hundreds.

5.2.1 Best-F1 Threshold

The threshold is read off the precision-recall curve at the point that maximises window-level F1, the harmonic mean of precision and recall. It balances the two at the level of individual windows and ignores the cost of each error.

5.2.2 Cost-Sensitive Threshold

We tested 200 candidate thresholds, spaced along the quantiles of the evaluation scores plus the best-F1 value. For each, we computed an event-level cost:

```
\text{Cost}(\tau) = N_{\text{missed}}(\tau) \times 400000 + N_{\text{false episodes}}(\tau) \times 2000
```

The threshold with the lowest cost was chosen. Ties go to the lowest mean detection delay, then to the highest threshold. A missed fault costs as much as 200 false alarms, which is why this threshold tends to favour catching every event.

5.3 Evaluation Metrics

Window level (each window is one example): precision, recall and F1.

Event level (what maintenance sees):

- **Fault-event recall:** the share of the four failure intervals in which at least one flagged window ends. Target at least 0.85.
- **False-alarm episodes per 24 h:** alarm episodes that never touch a failure interval, divided by the evaluated hours, times 24. Target under 2.
- **Detection delay:** time from the start of a failure interval to the end of the first flagged window inside it. Target under 5 minutes on average.
- **Estimated cost:** the formula of Section 5.2.2 applied to the chosen threshold.

The false-alarm count is in episodes, not windows, and the evaluated time is the span of the scored windows (about 184 days).

5.4 Precision–Recall Curves

The precision-recall curves, with the best-F1 point and the cost-aware point marked on each, are shown in Section 6.3 (Figures 6.1 and 6.2), because they come from the real run.

6 Experimental Results and Discussion

6.1 Experimental Status and Reporting Rule

All results below come from one complete run of the pipeline on the full MetroPT-3 dataset in Google Colab (seed 42, both models trained for 20 epochs). Every number is copied from the run's output files (metrics_summary.json, model_comparison.csv, cost_analysis.csv and experiment_report.md). Nothing was estimated or adjusted. The run was done once, so there are no repeat runs or confidence intervals.

6.2 Results Table

Table 6.1. Cost-aware operating point (event level)

Model	Threshold	Events detected	False-alarm episodes	False alarms / 24 h	Mean delay (min)	Estimated cost
Dense AE	46.50	4 of 4	196	1.07	50.40	₹3,92,000
LSTM AE	34.73	4 of 4	226	1.23	73.45	₹4,52,000

Table 6.2. Best-F1 operating point

Model	Threshold	Precision	Recall	F1	Events detected	False alarms / 24 h	Mean delay (min)
Dense AE	42.25	0.568	0.953	0.711	4 of 4	1.54	50.03
LSTM AE	28.51	0.546	0.946	0.692	4 of 4	3.77	42.99

Table 6.3. Window-level confusion counts at the cost-aware threshold

Model	True positive	False positive	False negative	True negative	Precision	Recall
Dense AE	8,238	6,916	1,788	417,081	0.544	0.822
LSTM AE	3,532	4,413	6,494	419,584	0.445	0.352

Table 6.4. Detection delay per failure at the cost-aware threshold (minutes)

Failure interval	Dense AE	LSTM AE
18 Apr 2020	30.25	46.12
29–30 May 2020	0.45	76.78
5–7 Jun 2020	0.40	0.40
15 Jul 2020	170.52	170.52

Table 6.5. Assignment targets at the cost-aware threshold

Target	Dense AE	LSTM AE
Fault-event recall of at least 0.85	1.00: met	1.00: met
Under 2 false alarms per 24 h	1.07: met	1.23: met
Mean detection delay under 5 min	50.40: not met	73.45: not met
No labelled fault data in training	met	met

The "within 3 readings" alternative delay condition applies only if sampling is slower than once a minute; we applied the 5-minute criterion.

6.3 Output Figures

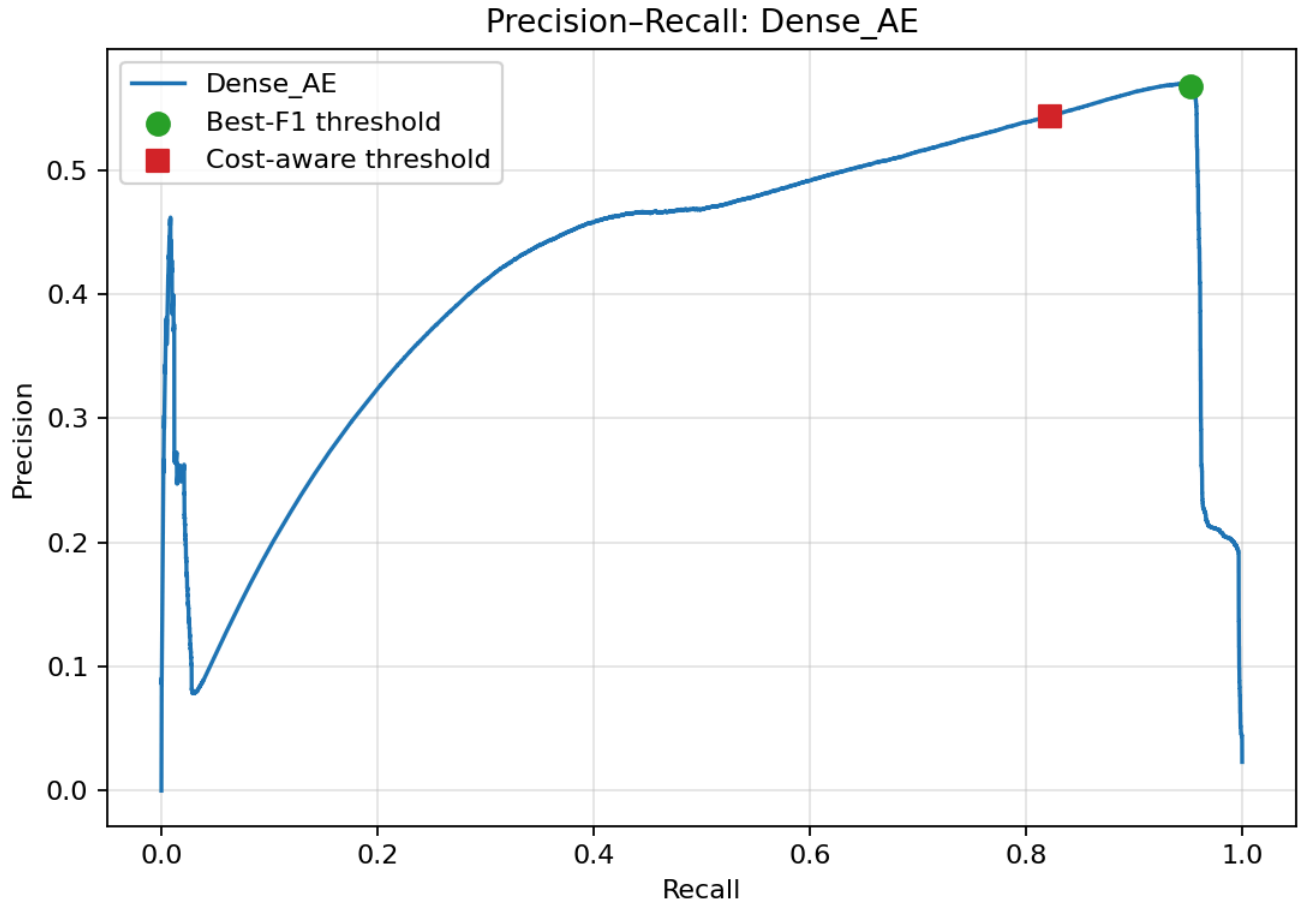


Figure 6.1. Precision-recall curve for the Dense AE. The green circle marks the best-F1 threshold and the red square the cost-aware threshold.

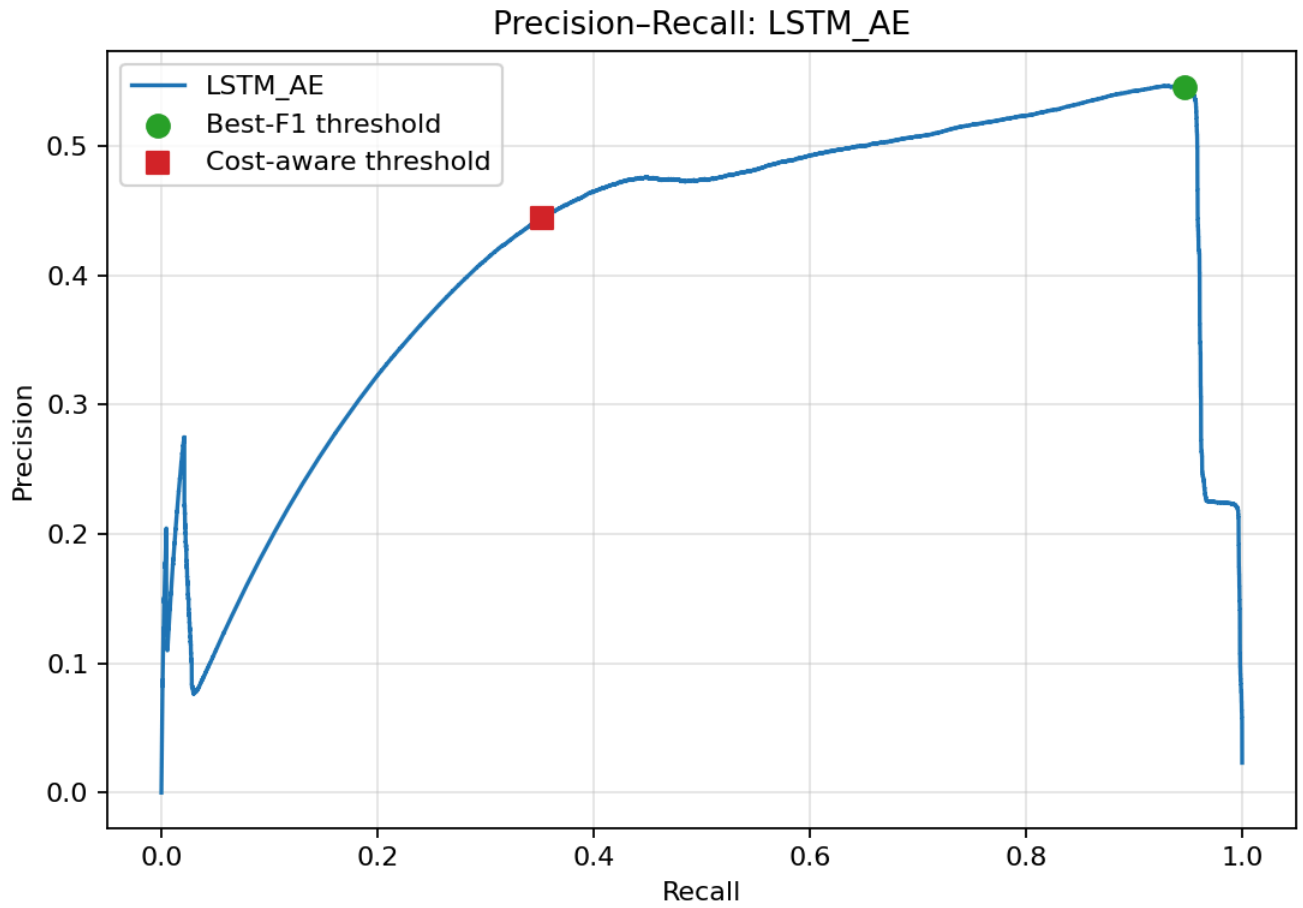


Figure 6.2. Precision-recall curve for the LSTM AE, with the same markers.

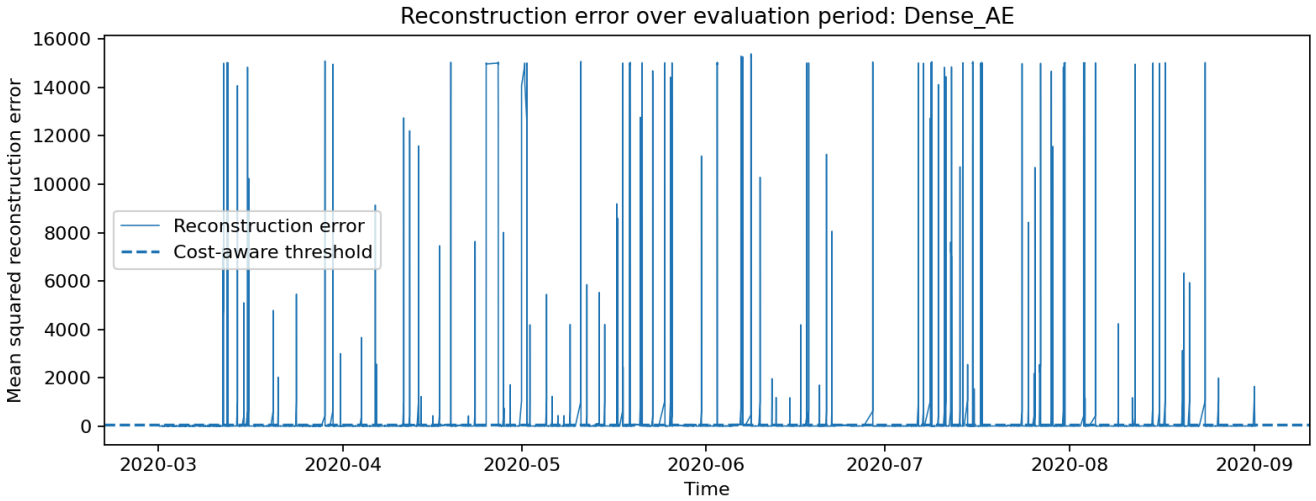


Figure 6.3. Reconstruction error of the Dense AE from March to September 2020. The dashed line is the cost-aware threshold.

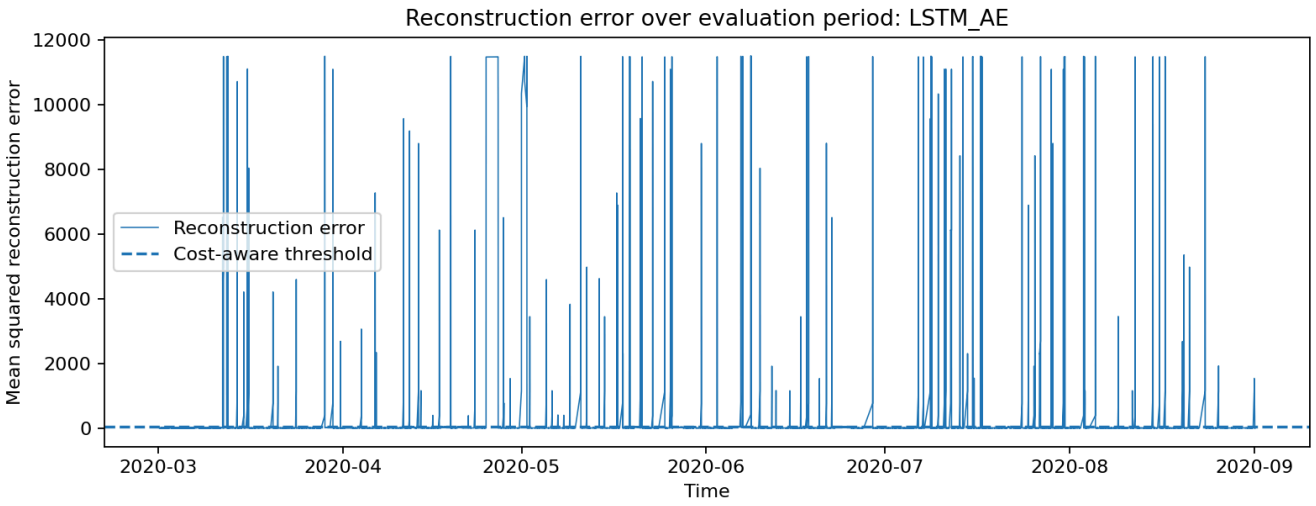


Figure 6.4. Reconstruction error of the LSTM AE over the same period.

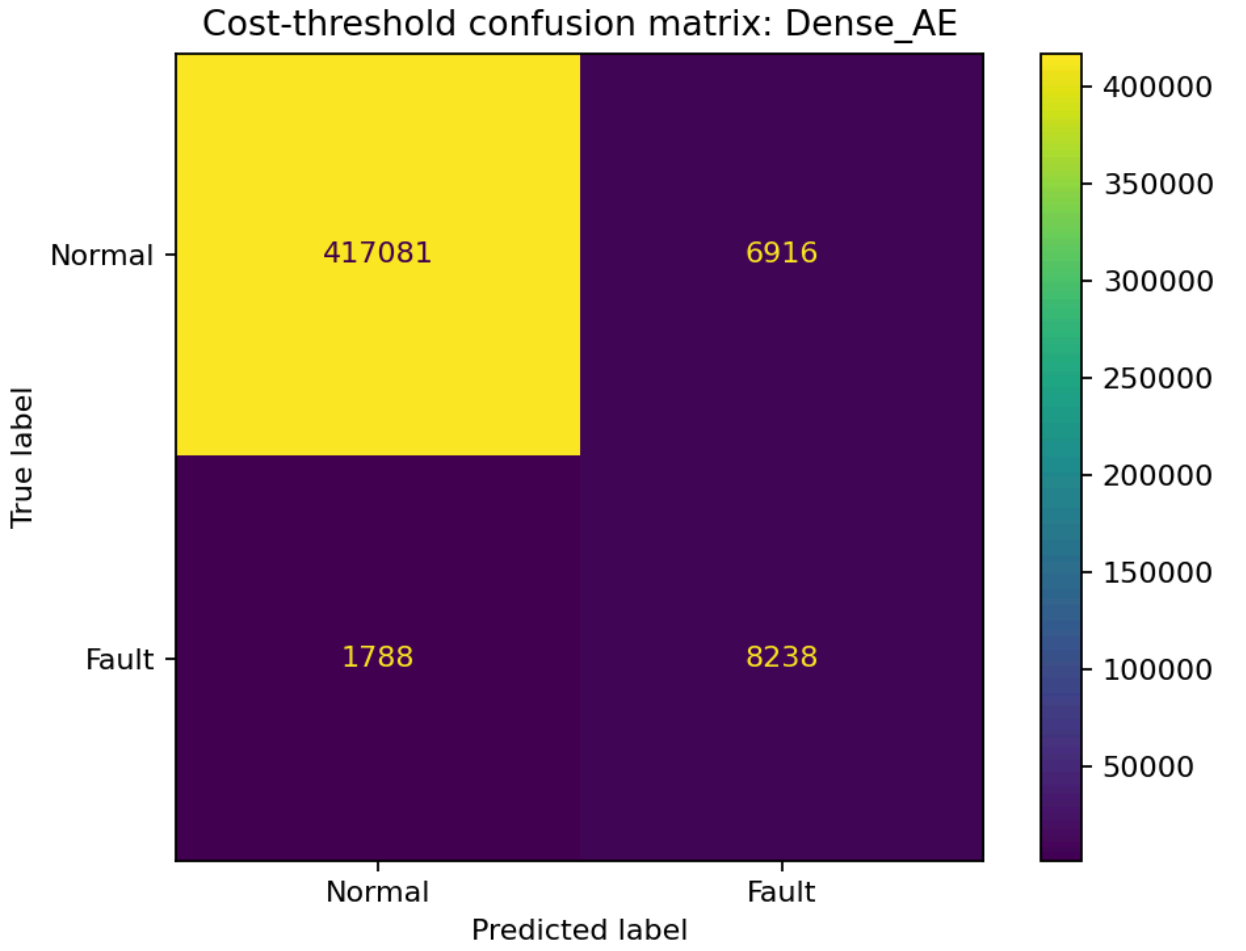


Figure 6.5. Window-level confusion matrix for the Dense AE at the cost-aware threshold.

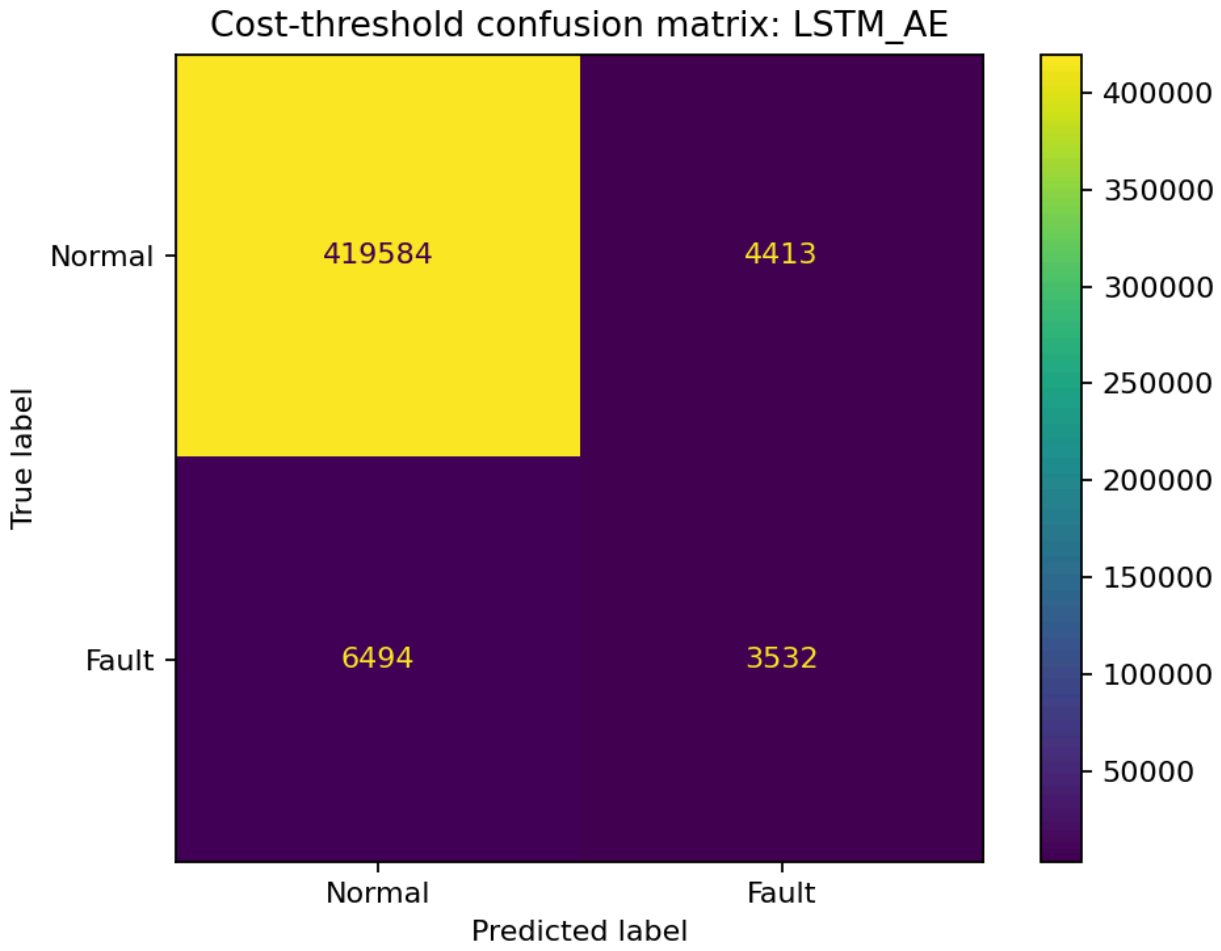


Figure 6.6. Window-level confusion matrix for the LSTM AE at the cost-aware threshold.

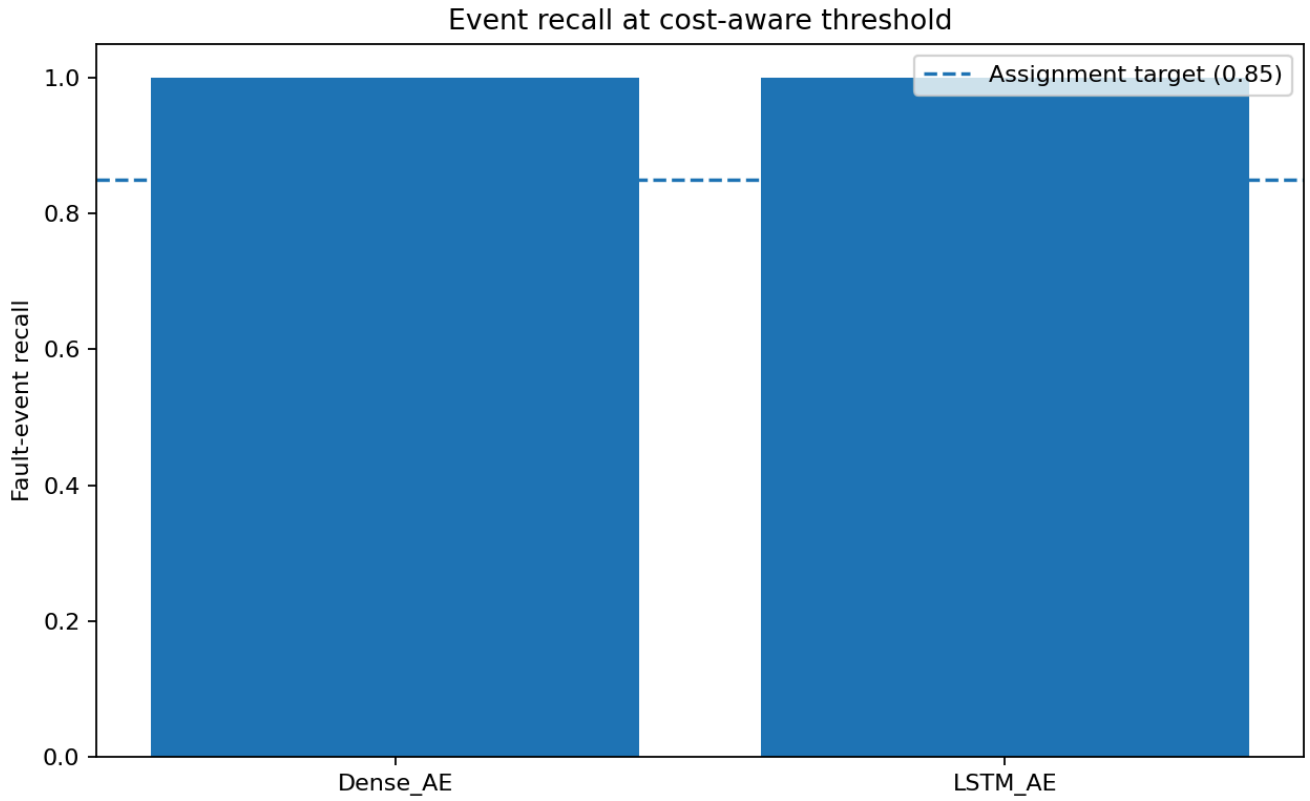


Figure 6.7. Event recall of both models against the 0.85 target.

6.4 Dense versus LSTM Comparison

At the cost-aware threshold the Dense model was better on false alarms (1.07 against 1.23 per 24 h), mean delay (50.4 against 73.5 minutes), cost (₹3,92,000 against ₹4,52,000) and window-level recall (0.82 against 0.35). It also had the higher best-F1 (0.711 against 0.692) and far fewer false alarms at that threshold (1.54 against 3.77 per 24 h). The LSTM was faster only at the best-F1 threshold, where its mean delay was 42.99 against 50.03 minutes. Both models detected all four failures at both thresholds.

So this experiment does not show a benefit from modelling the sequence. One likely reason is that the LSTM is under-trained: its final training loss was about twice that of the Dense model (0.166 against 0.081) and it used all 20 epochs. We did not train longer, so we cannot say whether it would catch up.

6.5 Discussion

Targets. Event recall and false-alarm rate were met by both models at the cost-aware threshold. The mean-delay target was missed by a wide margin, mainly because of the 15 July failure, which both models flagged 170.5 minutes after it began, and the 18 April failure (30 to 46 minutes). The Dense model flagged two failures within 0.45 minutes of their start. A very short delay may mean an alarm was already active when the interval began; we did not separate these cases.

Event recall is a weak test. Detecting an event needs only one flagged window inside a failure interval of 4.5 to 52.5 hours, so recall reached 1.00 even for the LSTM, which flagged only 35% of fault windows. The window-level recall is the better indicator of how persistently each model sees a fault.

False alarms. About 46% of flagged windows at the cost-aware threshold lie outside the four labelled intervals (Dense precision 0.544). The labels cover only reported failures, so some may be genuine anomalies that were never reported; the data cannot tell us. Figures 6.3 and 6.4 show isolated spikes reaching thousands, against thresholds of 46 and 35. The two models spike at almost the same moments, which points to unusual input data rather than a model fault. A plausible but untested explanation is a channel that barely varied in February, so that a small change later becomes a very large scaled value. Checking this per sensor is the natural next step.

Threshold choice. For the Dense model the cost-aware threshold lowered false-alarm episodes from 284 to 196 and window recall from 0.953 to 0.822, while still detecting every event. Since both thresholds detect all four events, the event-level cost is just false-alarm episodes times ₹2,000: for the Dense model ₹5,68,000 at best-F1 against ₹3,92,000 at the cost-aware threshold, and for the LSTM ₹13,88,000 against ₹4,52,000.

6.6 Limitations

- **Tuning on the test period:** thresholds were chosen and reported on the same labelled period, so results are optimistic. A separate calibration period and an untouched test period would be fairer.
- **Four events only:** each event is worth 0.25 of recall, so one miss would drop recall from 1.00 to 0.75.
- **Single run:** one seed, no repeats and no confidence intervals.
- **Delay resolution:** windows advance 3 readings at a time and delay is measured at the window end, so delays are resolved only to about one step.

- **Simplified cost model:** a flat cost per missed event and per false-alarm episode, ignoring severity and inspection scheduling.
- **Normal-period assumption:** February 2020 is assumed to be fault-free.

7 Conclusion and Future Scope

7.1 Conclusion

We built an unsupervised early-warning pipeline for a metro air compressor that learns only from normal February 2020 data. On the full MetroPT-3 evaluation period, both a Dense and an LSTM autoencoder detected all four reported air-leak failures with about one false alarm per day, meeting the recall (at least 0.85) and false-alarm (under 2 per 24 h) targets. Neither met the 5-minute detection-delay target: mean delay was 50.4 minutes for the Dense model and 73.5 for the LSTM.

The simpler Dense autoencoder performed better overall, with lower cost (₹3,92,000 against ₹4,52,000), fewer false alarms and a higher window-level recall. The LSTM showed no advantage here, possibly because it was under-trained. The cost-aware threshold cut false-alarm episodes by about 31% for the Dense model (284 to 196) and 67% for the LSTM (694 to 226) compared with the best-F1 threshold, without losing any event.

These results are a first assessment. Thresholds were tuned on the same period used for reporting and only four failures exist, so performance on new data is likely to be lower.

7.2 Future Scope

- **Proper validation:** tune thresholds on a separate calibration period and report on an untouched test period.
- **Investigate the spikes:** trace the large reconstruction-error spikes to individual sensors to separate glitches, start-up transients and real degradation.
- **Reduce delay:** require alarms to persist over several windows, use shorter windows or smaller strides, and add trend features.
- **Better LSTM:** train longer and tune its size and learning rate before concluding anything about sequence modelling.
- **Per-sensor attribution:** show which channels drive each alarm so engineers can locate the fault.
- **Repeat runs:** several seeds with confidence intervals.

- **Deployment:** stream scoring with the saved scaler and models, and periodic recalibration.

Appendix A Reproducibility and Implementation Record

A.1 Execution Checklist

- [ ] Open DL_T19_3E_Colab.ipynb in Google Colab and choose a GPU runtime.
- [ ] Run all cells: the notebook writes config.json, download_data.py, the src modules and run_project.py.
- [ ] download_data.py fetches MetroPT-3 from UCI (about 208 MB).
- [ ] run_project.py trains both models and writes everything to outputs/.
- [ ] Download outputs.zip and keep metrics_summary.json, model_comparison.csv, cost_analysis.csv and experiment_report.md with the report.

Key configuration (config.json): training 2020-02-01 to 2020-03-01, sequence length 30, stride 3, validation fraction 0.20, 20 epochs, batch size 256, seed 42, costs ₹4,00,000 per missed fault and ₹2,000 per false alarm.

A.2 High-Level Processing Pseudocode

```
load CSV, sort by timestamp, drop duplicates and missing values
train  = rows in [2020-02-01, 2020-03-01)
fit, val = first 80%, last 20% of train (time order)
scaler = StandardScaler fitted on fit only
X_fit, X_val = windows(scale(fit)), windows(scale(val))   # length 30, stride
3
eval   = rows from 2020-03-01 onward
y_eval = window is 1 if it overlaps a failure interval
X_eval = windows(scale(eval))
for model in [DenseAE, LSTMAE]:
    train model on X_fit, validate on X_val (MSE, Adam, early stopping)
    score = mean squared reconstruction error of each eval window
    th_f1   = threshold maximising F1 on the precision-recall curve
    th_cost = candidate threshold with lowest
              missed_events*400000 + false_alarm_episodes*2000
    for th in [th_f1, th_cost]:
        flag windows with score >= th, merge into alarm episodes
        report event recall, false alarms per 24 h, detection delay, cost
```

A.3 Result Substitution Notes

No placeholders remain in this report. Where each result comes from:

Reported item	Source file
Thresholds, precision, recall, F1, delays, cost	metrics_summary.json and model_comparison.csv
Cost table	cost_analysis.csv
Per-event delays and target checks	experiment_report.md
Figures 6.1 to 6.7	precision_recall_*.png, *reconstruction_error_*.png, confusion_matrix_*.png, model_comparison.png

The cost-aware threshold selection in src/metrics.py breaks cost ties by lowest mean detection delay, a rule added by us. Trained models are saved as dense_ae.keras and lstm_ae.keras.

Team Contributions

Roll number	Name	Assigned role	Work carried out
23261A6648	Srihari Harkar	Data Lead	To be completed by the team
23261A6626	M. Tharun Reddy	Model Lead	To be completed by the team
23261A6602	A. Prabhas	Evaluation and Report Lead	To be completed by the team

Each member must fill in the work they actually did before submission.

AI Tool Usage Disclosure

- **Tool:** Claude (Anthropic).
- **Code:** Claude wrote the four project modules that were missing from the repository (data loading and windowing, the two autoencoder models, metrics and threshold selection, report generation), made two small changes to run_project.py (both thresholds marked on the precision-recall plots, and a delay-based tie-break in the cost-aware threshold), and assembled the Colab notebook. Its code was checked on synthetic data for correct execution only.
- **Experiments:** run by the team on Google Colab. All results in this report come from those output files.
- **Writing:** Claude drafted this report, the technical report and the plain-language note for plant engineers from the run outputs. The literature review was written from general knowledge of the cited works and should be checked against the sources.
- **Responsibility:** the team has reviewed the work and can explain it.

References

1. Davari, N., Veloso, B., Ribeiro, R. P., and Gama, J., "MetroPT-3 Dataset," UCI Machine Learning Repository, DOI: 10.24432/C5VW3R, 2023.
2. Davari, N., Veloso, B., Ribeiro, R. P., Pereira, P. M., and Gama, J., "Predictive maintenance based on anomaly detection using deep learning for air production unit in the railway industry," 2021 IEEE 8th International Conference on Data Science and Advanced Analytics.

3. Chandola, V., Banerjee, A., and Kumar, V., "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, 2009.
4. Hochreiter, S., and Schmidhuber, J., "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
5. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, 2016, Chapter 14.
6. Hinton, G. E., and Salakhutdinov, R. R., "Reducing the dimensionality of data with neural networks," Science, vol. 313, no. 5786, pp. 504–507, 2006.
7. Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., and Shroff, G., "LSTM-based encoder-decoder for multi-sensor anomaly detection," ICML 2016 Anomaly Detection Workshop, arXiv:1607.00148.